MODULE5

FastAPI Fundamentals

1. What is an API?

Definition: API stands for Application Programming Interface. It allows two software applications to communicate with each other.

Example: A weather application uses an API to get weather information from a server.

2. What is FastAPI?

Definition: FastAPI is a Python framework used to build web applications and REST APIs.

Advantages:

- Easy to create APIs.
- Fast and efficient.
- Provides automatic API documentation.
- Supports input validation.

3. What is an endpoint?

An endpoint is a URL where an API provides a particular operation.

Example: `/students` is an endpoint for student-related information.

4. HTTP methods

Method	Purpose	Example
GET	Read data	View students
POST	Add data	Add a student

PUT	Update data	Update student marks
DELETE	Delete data	Remove a student

SQLite Database

1. What is a database?

Definition: A database is an organized collection of information stored so that it can be accessed, managed, and updated easily.

Example: A student database contains student IDs, names, and marks.

2. What is SQLite?

Definition: SQLite is a lightweight database that stores information in a file. Python includes the `sqlite3` module, so no separate database server is required.

Create a database and table :-

```
import sqlite3

connection = sqlite3.connect("students.db")
cursor = connection.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   marks REAL NOT NULL
)
""")

connection.commit()
connection.close()

print("Database created successfully")
```

OUTPUT:-

Database created successfully

What is SQL?

Definition: SQL stands for Structured Query Language. It is used to create, read, update, and delete information in a database.

Example:

SELECT * FROM students;

CRUD Operations

CRUD stands for Create, Read, Update, and Delete.

These operations are used to manage database records.

1. Create — Add a student

import sqlite3connection = sqlite3.connect("students.db")connection.execute("INSERT INTO students (name, marks) VALUES (?, ?)", ("Pujitha", 90))connection.commit()connection.close()print("Student added successfully")

Output:

Student added successfully

2. Read — Display students

import sqlite3connection = sqlite3.connect("students.db")records = connection.execute("SELECT * FROM students").fetchall()for student in records: print(student)connection.close()

Example output:

(1, 'Pujitha', 90.0)

3. Update — Change marks

import sqlite3connection = sqlite3.connect("students.db")connection.execute("UPDATE students SET marks = ? WHERE id = ?", (95, 1))connection.commit()connection.close()print("Marks updated successfully")

Output:

Marks updated successfully

4. Delete — Remove a student

import sqlite3connection = sqlite3.connect("students.db")connection.execute("DELETE FROM students WHERE id = ?",   (1,))connection.commit()connection.close()print("Student deleted successfully")

Output:

Student deleted successfully

Important: Run the examples in order when practising with the same database. The update and delete examples use student ID 1, which must exist.

Authentication, Validation, and API Testing

What is authentication?

Definition: Authentication checks whether a user or application has valid credentials before allowing access.

Example: Entering a username and password to log in.

What is schema validation?

Definition: Schema validation checks whether the input data has the correct format and values.

For example, student marks should be between 0 and 100.

FastAPI uses Pydantic models for data validation.

from pydantic import BaseModel, Fieldclass Student(BaseModel):   name: str   marks: float = Field(ge=0, le=100)

Here, `name` must be text and `marks` must be a number between 0 and 100.

What is API testing?

Definition: API testing checks whether endpoints work correctly and return the expected results.

Mini Project — Student Management REST API

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import sqlite3

app = FastAPI(title="Student Management API")
```

```
class Student(BaseModel):
    name: str = Field(min_length=1, max_length=80)
    marks: float = Field(ge=0, le=100)


def connect_db():
    connection = sqlite3.connect("students.db")
    connection.row_factory = sqlite3.Row
    return connection


def create_table():
    with connect_db() as connection:
        connection.execute("""
            CREATE TABLE IF NOT EXISTS students (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                marks REAL NOT NULL
            )
        """)


@app.on_event("startup")
def startup():
    create_table()


@app.get("/")
def home():
    return {"message": "Student Management API is running"}


@app.post("/students", status_code=201)
def add_student(student: Student):
    with connect_db() as connection:
        cursor = connection.execute(
            "INSERT INTO students (name, marks) VALUES (?, ?)",
            (student.name, student.marks)
        )
        student_id = cursor.lastrowid
        record = connection.execute(
            "SELECT * FROM students WHERE id = ?",
            (student_id,)
```

```
        ).fetchone()

    return dict(record)


@app.get("/students")
def get_students():
    with connect_db() as connection:
        records = connection.execute(
            "SELECT * FROM students ORDER BY id"
        ).fetchall()

    return [dict(record) for record in records]


@app.get("/students/{student_id}")
def get_student(student_id: int):
    with connect_db() as connection:
        record = connection.execute(
            "SELECT * FROM students WHERE id = ?",
            (student_id,)
        ).fetchone()

    if record is None:
        raise HTTPException(
            status_code=404,
            detail="Student not found"
        )

    return dict(record)


@app.put("/students/{student_id}")
def update_student(student_id: int, student: Student):
    with connect_db() as connection:
        cursor = connection.execute(
            "UPDATE students SET name = ?, marks = ? WHERE id = ?",
            (student.name, student.marks, student_id)
        )

        if cursor.rowcount == 0:
            raise HTTPException(
                status_code=404,
                detail="Student not found"
```

```
        )

        record = connection.execute(
            "SELECT * FROM students WHERE id = ?",
            (student_id,)
        ).fetchone()

    return dict(record)


@app.delete("/students/{student_id}")
def delete_student(student_id: int):
    with connect_db() as connection:
        cursor = connection.execute(
            "DELETE FROM students WHERE id = ?",
            (student_id,)
        )

        if cursor.rowcount == 0:
            raise HTTPException(
                status_code=404,
                detail="Student not found"
            )

    return {"message": "Student deleted successfully"}
```